How Single Sign-On Works

From a password box to OpenID Connect, one problem at a time

Seven stages. Each one starts from the previous design, gets attacked, and adds exactly the one value needed to survive that attack. By the end you will have derived the authorization code flow with PKCE, and the OpenID Connect layer on top of it, rather than been handed either.

Contents

1. How to read this — the colour code

Every stage of this story is really about **one question**: who is allowed to see this particular value? Almost every security property in SSO follows from that, and almost every historical vulnerability came from a value reaching someone it should not have.

So the values in the diagrams are coloured by *audience* rather than by what they technically are. Learn these five and the flows read themselves.

Colour	Group	Examples	Who may ever see it
Red	**User credentials**	`password`, TOTP code	**Only the identity provider.** The crown jewels. If any other party sees this, the whole model has failed.
Green	**Session state**	`session_id`, session cookie	One server and the user's browser. Says "this browser already proved who it is".
Purple	**Application identity**	`client_id`, `client_secret`, registered `redirect_uri`	Identifies and authenticates the *application*, not the user. The id is public; the secret is not, and only a server can hold one.
Amber	**Bearer tokens**	`access_token`, `refresh_token`	"Bearer" is literal: whoever holds it can use it. Treated as a credential in transit and kept short-lived.
Blue	**Single-use flow plumbing**	`code`, `state`, `code_verifier`, `code_challenge`	Exists only for the seconds a login takes. Each one binds two halves of a flow together so they cannot be mixed and matched by an attacker.

THE ONE SENTENCE TO KEEP

Every stage below adds a value because a *specific attack* was possible without it. If you can name the attack, you can always re-derive why the value exists — which is far more durable than memorising the parameter list.

2. Stage 1 — One app, one password

STAGE 1

A single application authenticates its own users

Introduces: **password**, **session_id**

Start with no SSO at all. One application, its own user table. Alice types an email and a password; the app compares the password against a stored hash and, if it matches, marks the browser as logged in.

The interesting part is the second half. The app does *not* ask for the password again on the next request — it issues a **session_id**: a long random string stored in a cookie. That is the first appearance of the central idea of this whole document:

IDEA 1

A credential is exchanged, once, for a **less powerful, shorter-lived stand-in**. The password proves identity; the session id merely proves that a password was checked recently. Every later stage is a more elaborate version of this same trade.

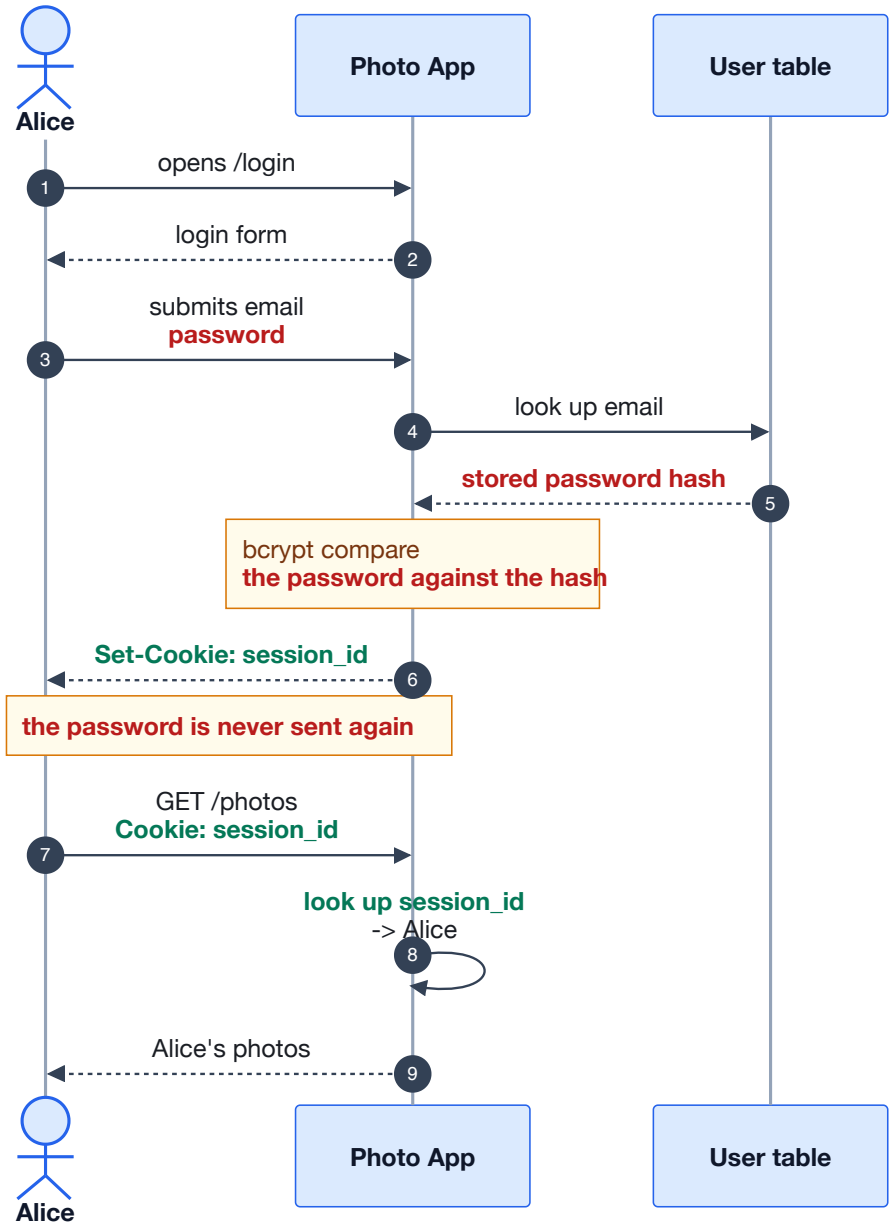

Figure 1 — Classic password login. The password (red) appears exactly once; everything after that runs on the session id (green).

Notable unhappy flow: a guessable session id

If **session_id** were sequential — `1001`, `1002` — an attacker would simply try other numbers and land inside someone else's account without ever touching a password. This is why session ids must come from a cryptographic random source with enough entropy to make guessing hopeless.

That constraint never goes away. Every blue and green value in this document has the same requirement, for the same reason. When you see later that a PKCE verifier must be at least 43 characters, it is this same rule reappearing.

3. Stage 2 — Two apps, and the temptation to share the password

STAGE 2

A second app appears, and asks for the first app's password

Introduces: nothing new — and that is exactly the problem

Now there is a second application, Print Shop, which wants to print Alice's photos. It needs access to her Photo App account. The obvious implementation is the wrong one, and seeing precisely *why* it is wrong is what motivates everything else.

Print Shop shows a form: "enter your Photo App email and password". It then replays those credentials against Photo App on Alice's behalf. It works perfectly. It is also a catastrophe.

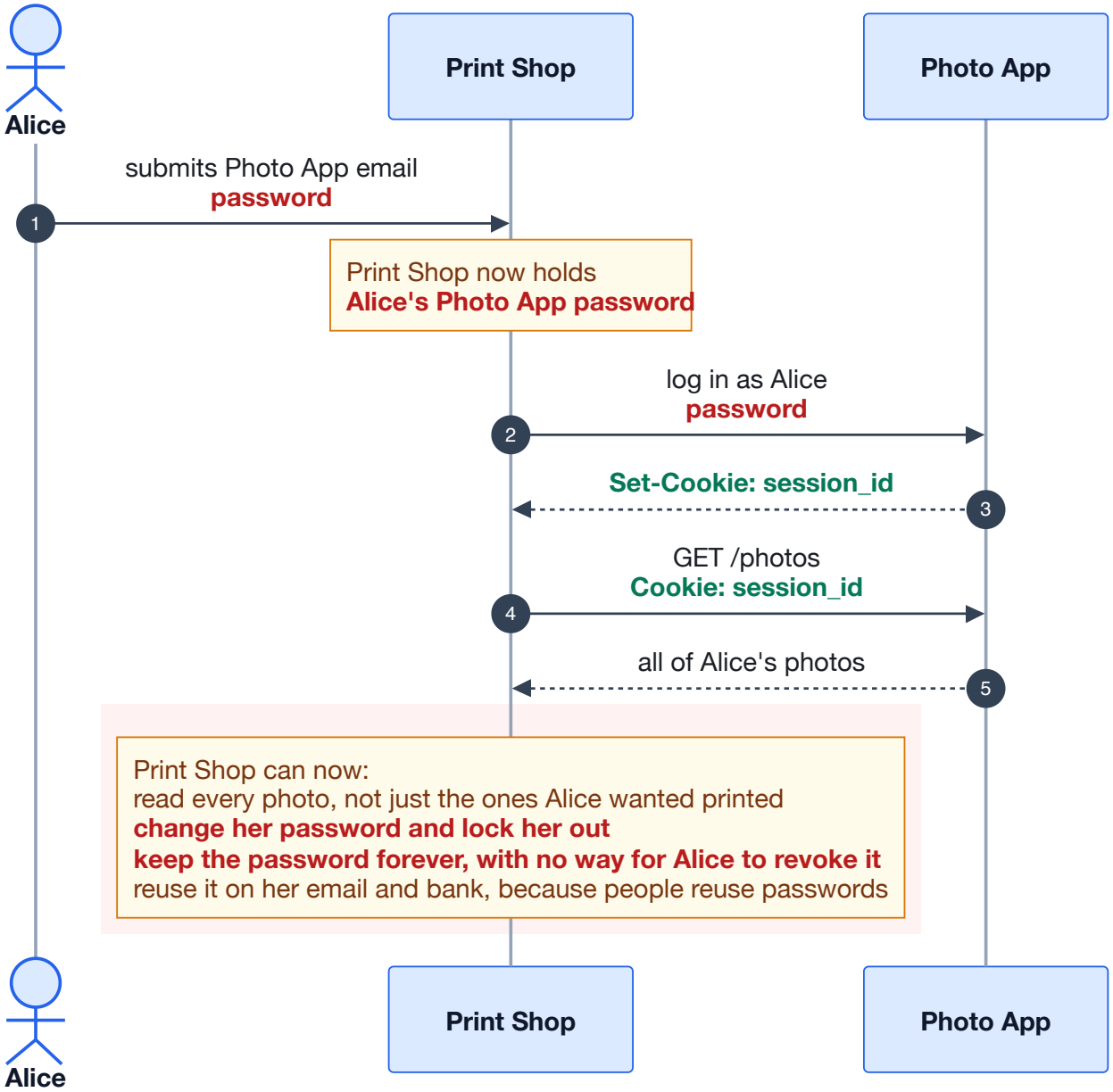

Figure 2 — Credential sharing. Functionally correct, and the reason delegated authorization had to be invented.

Why this is unfixable by being careful

Four problems, and none of them are solved by Print Shop being well-intentioned:

- **No scope.** A password is all-or-nothing. There is no way to say "may read photos, may not delete them".
- **No revocation.** Alice cannot withdraw Print Shop's access without changing her password, which also signs her out everywhere else.
- **Unbounded blast radius.** Print Shop's database is now as sensitive as Photo App's. Every integration multiplies the number of places the password can leak from.
- **It trains users to be phished.** If legitimate sites ask for another site's password, users cannot tell a real request from a fake one.

> THE RULE THAT FOLLOWS
>
> **The password must only ever be typed into the identity provider itself.** No third-party application may see it — not in transit, not in memory, not for a moment. Everything from here on exists to satisfy this rule while still letting Print Shop do its job.

And that rule has an immediate mechanical consequence. If Alice must type her password into Photo App's own page, then her *browser* has to go to Photo App, and come back. The redirect is not an implementation detail chosen for convenience — it is forced on us by the rule above.

4. Stage 3 — Never let the app see the password: redirects

> STAGE 3
>
> **Send the user to the identity provider; get a token back**
>
> Introduces: **access_token**, **redirect_uri**

Photo App becomes an identity provider. Print Shop no longer collects credentials at all. Instead it sends Alice's browser to Photo App with a note saying "when you are done, send her back here". Alice authenticates on Photo App's own domain, sees Photo App's own URL in the address bar, and Photo App redirects her back carrying an **access_token**.

Print Shop uses that token to call Photo App's API. It never sees the password. Notice what has been gained: the token can be scoped to "read photos", it can expire, and Alice can revoke it without touching her password.

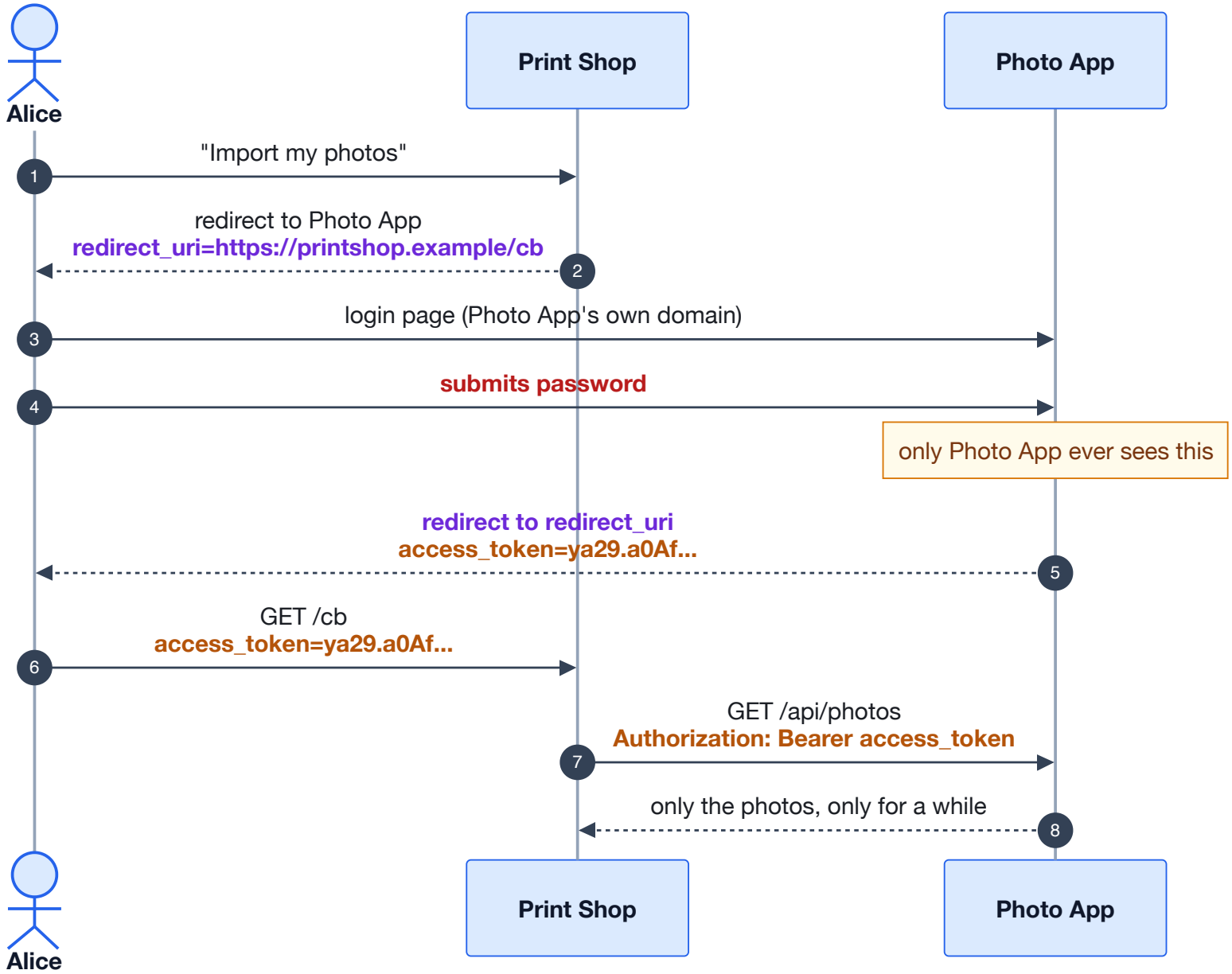

Figure 3 — The redirect model. The password stays inside the identity provider; a scoped, expiring token crosses the boundary instead.

Notable unhappy flow: the open redirect — and it is devastating

Look closely at step 2. Print Shop told Photo App where to send the token, and Photo App simply believed it. Nothing stops an attacker from crafting a link that starts a real login against the real Photo App, but names *their own* server as the destination.

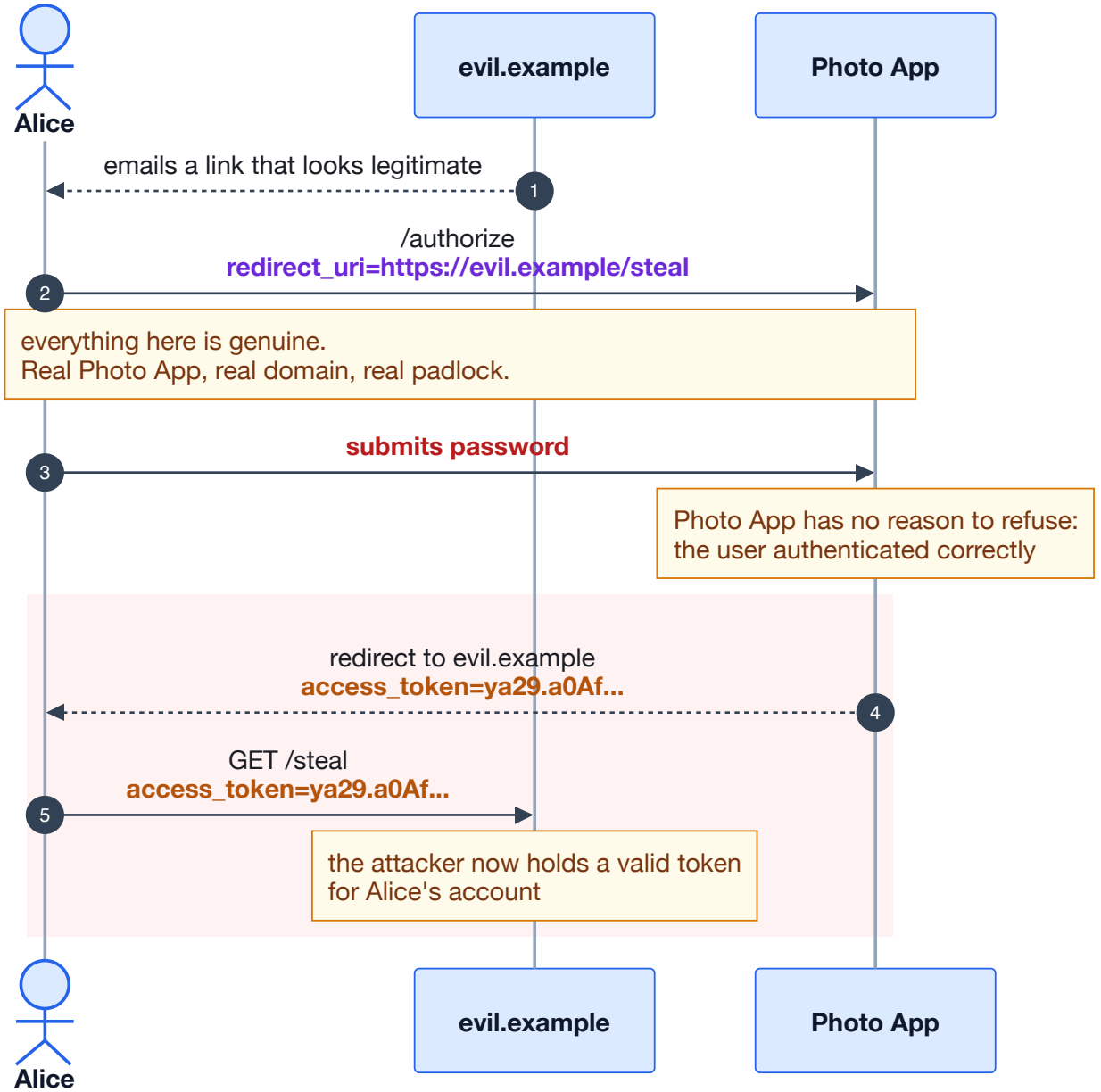

Figure 4 — Token exfiltration through an attacker-supplied redirect_uri. Alice did nothing wrong and saw nothing suspicious.

The fix is not to validate the URL cleverly. It is to **refuse to accept the destination from the request at all**. The application registers its **redirect_uri** with the identity provider in advance, and at login time the value sent must match the registered one exactly.

> **WHY "EXACTLY" IS NOT PEDANTRY**
>
> Matching by prefix or by domain has repeatedly proven exploitable — an open redirector anywhere on the registered domain turns into token theft, and a trailing-path match lets an attacker append their own. Exact string comparison is the only rule with no known bypass, and it is why a stray trailing slash in your configuration will be rejected outright. That strictness is a feature.

But registering the redirect uri raises a new question. Photo App now needs to know *which* registration to check against — that is, which application is asking.

5. Stage 4 — Naming the app, and proving the round trip

> STAGE 4
>
> **Identify the application, and tie the response to the request**
>
> Introduces: **client_id**, **state**

client_id is the answer to "which application is asking". It is a public identifier — it appears in URLs and in browser history, and that is fine, because on its own it grants nothing. Its job is to select the registration that holds the allowed **redirect_uri**.

With those two, the flow is meaningfully safe against the previous attack. But one gap remains, and it is a subtle one: **nothing ties the response Print Shop receives to a request Print Shop actually started.**

Notable unhappy flow: login CSRF

An attacker begins a login for *their own* Photo App account, stops at the point where they receive their token, and then tricks Alice's browser into visiting the callback URL carrying it. Alice's Print Shop session silently becomes linked to the attacker's Photo App account. She then uploads her private photos into what is, unknowingly, the attacker's storage.

The defence is **state**: a random value the client generates, stashes locally, sends with the request, and compares on return. A callback carrying a state the client never issued is discarded.

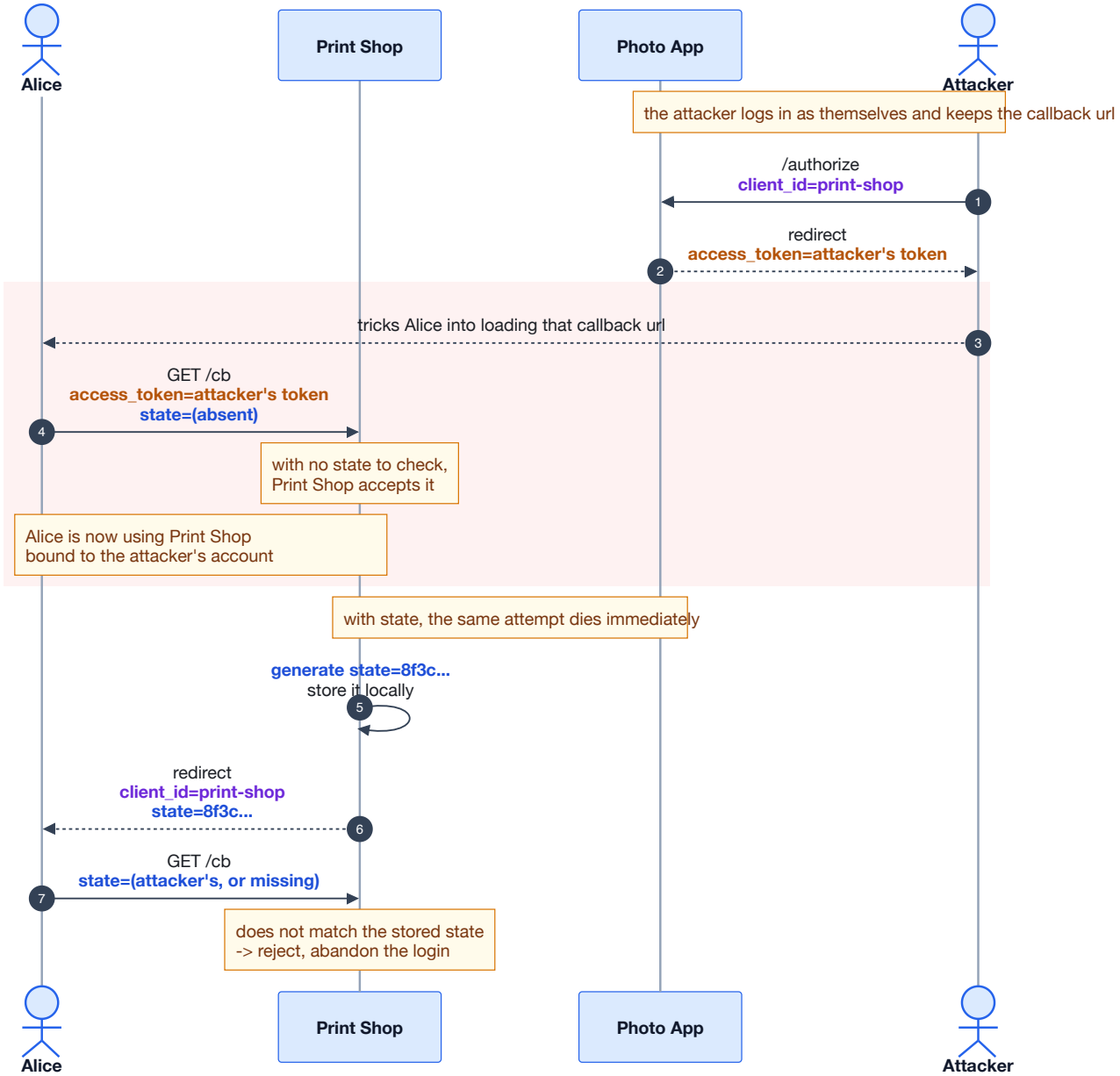

Figure 5 — Login CSRF, and how `state` stops it. The check is "did I start this?", not "is this user valid?".

> **A USEFUL DISTINCTION**
>
> **state** protects the *client* from accepting a response it did not ask for. It is not about proving anything to the identity provider. Keep that separate in your head from PKCE, which is about proving something *to* the identity provider — they are often confused because both are random blue values generated by the client.

The problem that remains: the token is in the URL

Stage 4 is roughly the old **implicit flow**, and it was the standard advice for browser applications for years. Its weakness is structural, and no amount of validation fixes it: the **access_token** is delivered *through the browser's address bar*. That means it can land in:

- browser history, and anything that syncs it,
- server access logs, if the URL is ever sent onward,
- `Referer` headers to third parties,
- any script running on the callback page, including injected ones.

And this is a token that may be valid for an hour and usable from anywhere. The next stage fixes this by never letting the real token touch the browser.

6. Stage 5 — Getting the token out of the browser

> STAGE 5
>
> **Return a useless one-time ticket; redeem it out of band**
>
> Introduces: **authorization code**, **client_secret**

The insight is to split the return trip in two. Through the browser, the identity provider sends only an **authorization code**: a short-lived, single-use ticket that is worthless on its own. Print Shop's *server* then exchanges that code for the real token over a direct HTTPS connection that the browser never sees.

What makes the code worthless to a thief is the second new value: the **client_secret**, shared between Print Shop's server and Photo App when the application was registered. Redeeming a code requires it.

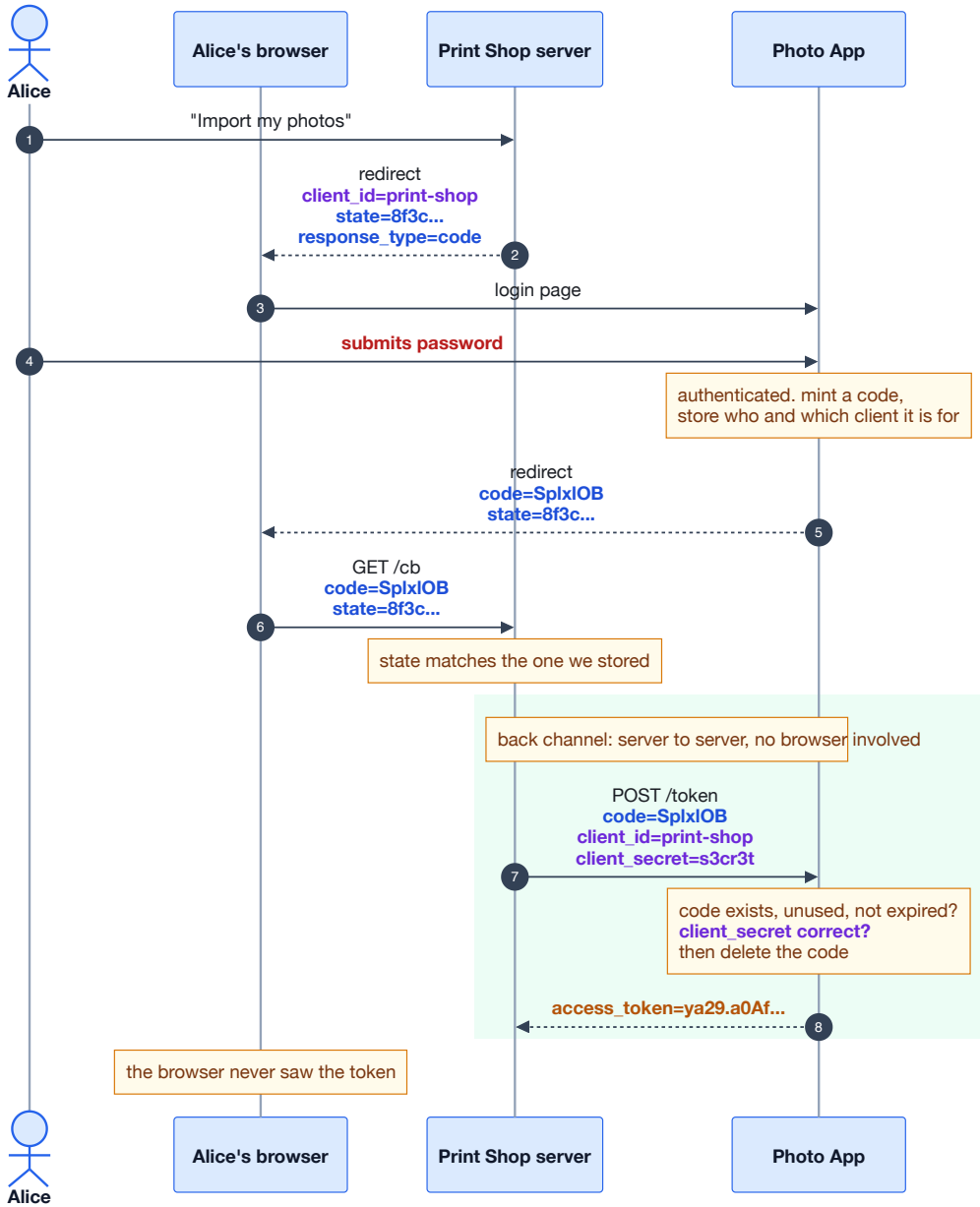

Figure 6 — The authorization code flow. Only the blue single-use code crosses the browser; the amber token travels server to server.

Notable unhappy flow: the code is stolen anyway

Suppose an attacker does capture the code — from a log, over the shoulder, via a malicious browser extension. They try to redeem it:

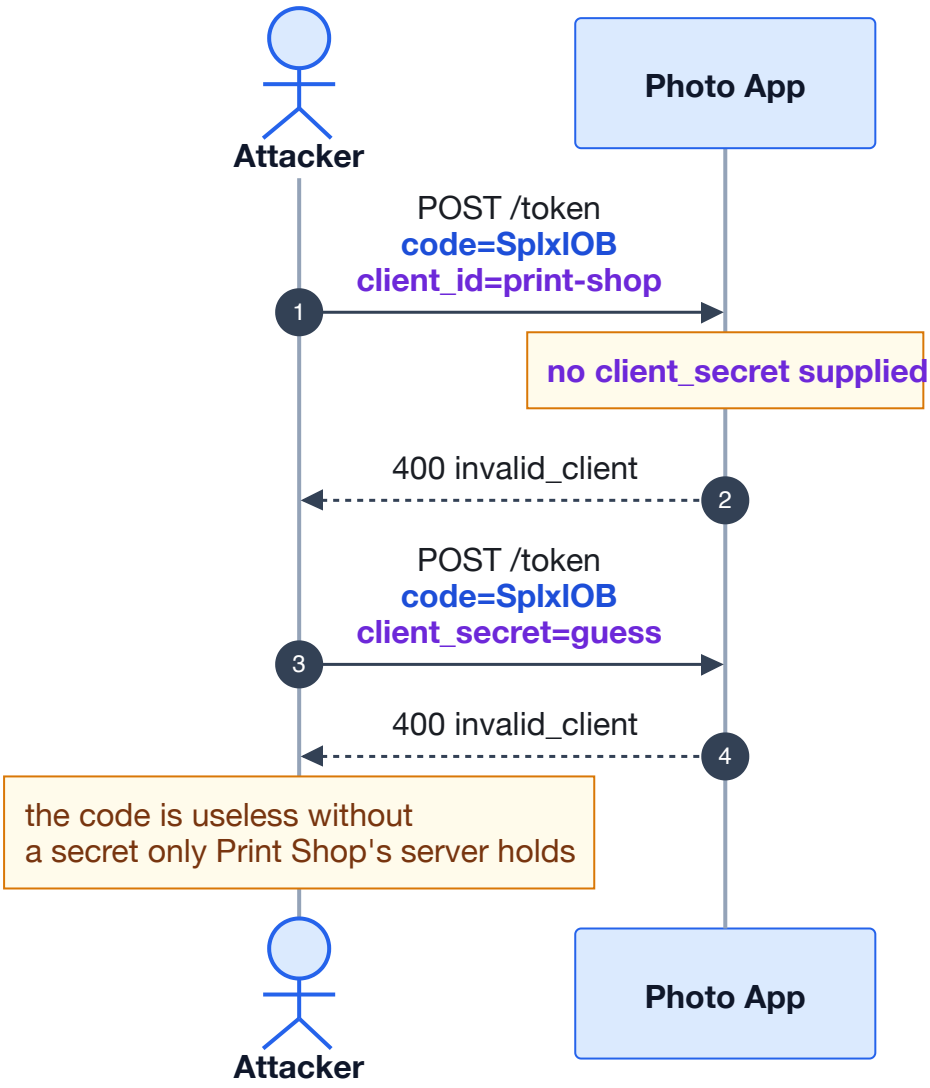

Figure 7 — A stolen code is inert. This is the property that makes the code flow safe to route through a browser.

Three further rules make this hold up, and each one has a reason:

- **Single use.** The code is deleted the moment it is redeemed. If one is ever presented twice, the honest party and an attacker have both used it — a strong signal to revoke everything issued from it.
- **Short lifetime.** Typically under a minute, because it only has to survive one redirect.
- **Bound to the same `redirect_uri`.** The value sent at the token step must match the one the code was issued for, so a code obtained through one registration cannot be redeemed as another.

AND NOW THE WALL

This entire design rests on **client_secret** being secret. That works for a server. It is **impossible** for a single-page app, a mobile app, or a desktop app: anything shipped to the user's device can be read by the user, and therefore by an attacker. Putting a secret in JavaScript is not a weaker version of this design — it is the same as having no secret at all, while pretending otherwise.

7. Stage 6 — When you cannot keep a secret: PKCE

> STAGE 6
>
> **Replace the long-lived shared secret with a per-login proof**
>
> Introduces: **code_verifier**, **code_challenge**

A public client cannot hold a secret that is the same every time, because that secret would have to ship with the app. But it *can* invent a brand-new secret for each individual login, keep it in memory, and prove it knew it — without ever transmitting it where an interceptor could catch it.

That is Proof Key for Code Exchange, and the mechanism is a one-way hash:

1. Before redirecting, the app generates a random **code_verifier** and keeps it locally.
2. It sends only `SHA-256(verifier)` — the **code_challenge** — with the authorization request.
3. The provider stores that challenge next to the code it issues.
4. At the token endpoint the app presents the original verifier. The provider hashes it and compares.

The challenge travels through the browser and may be observed. That is harmless: SHA-256 cannot be reversed, so seeing the challenge tells an attacker nothing about the verifier. The verifier travels only on the direct back-channel call, together with the code.

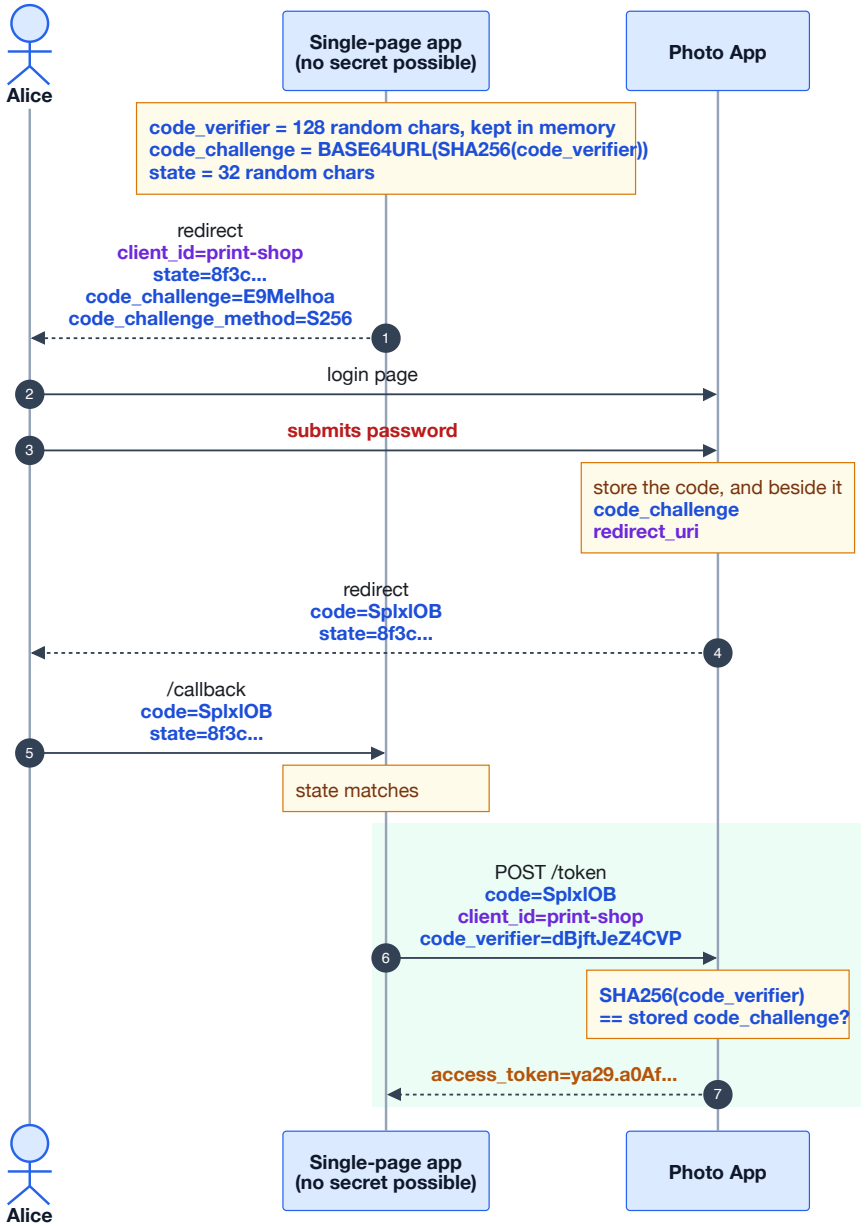

Figure 8 — Authorization code with PKCE. No client secret anywhere, and the code is still useless to anyone who intercepts it.

Notable unhappy flow: code interception on a public client

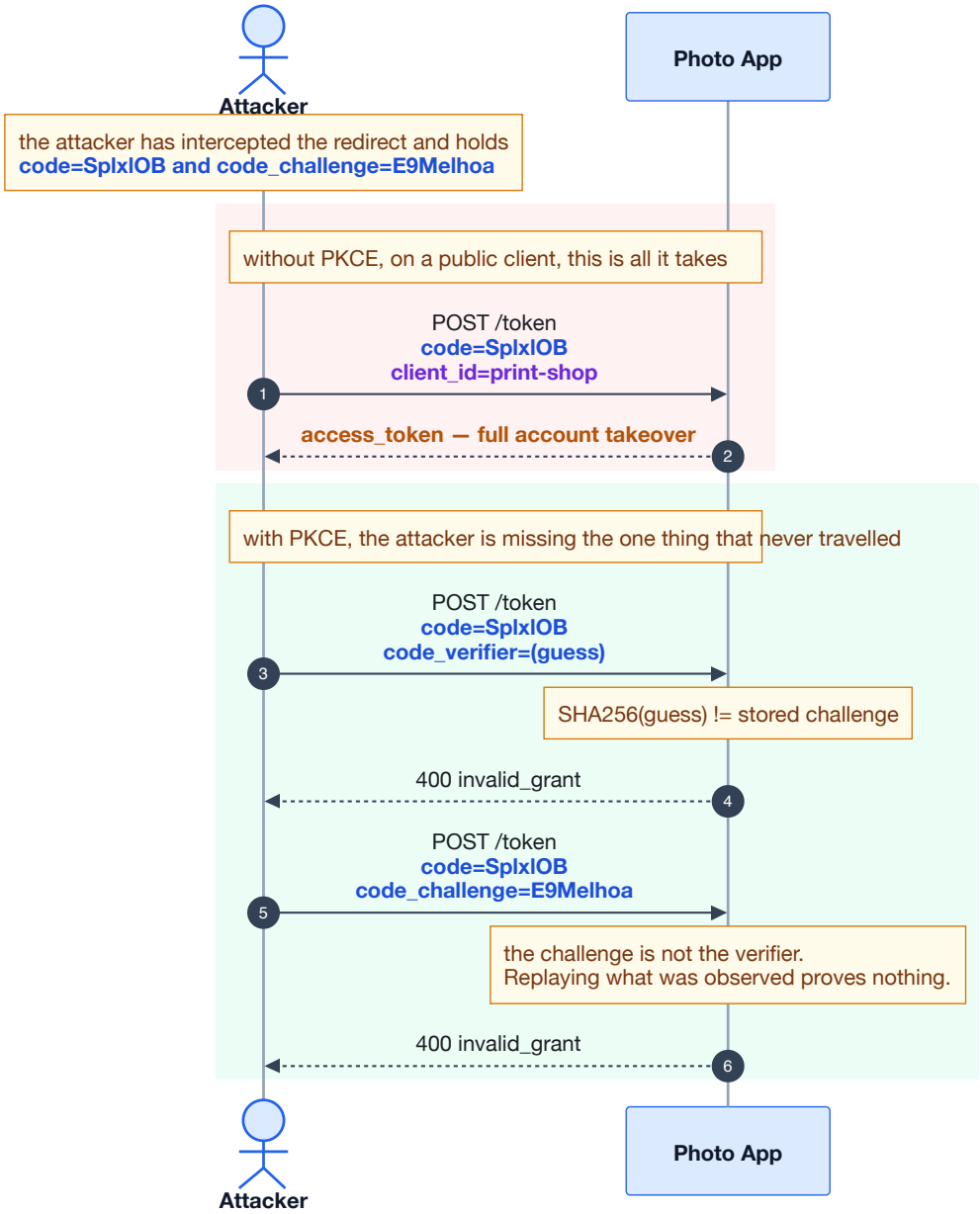

Figure 9 — The same interception, with and without PKCE. Note the second attempt: replaying the observed challenge does not work, which is the whole point of hashing.

Three details that look fussy and are not

- **Only `S256`, never `plain`.** The spec also allows sending the verifier itself as the challenge. That would put the verifier through the browser — exactly the exposure PKCE exists to avoid — so an attacker who saw the authorization request would have everything.
- **43–128 characters.** A short verifier could be brute-forced from the challenge, since the attacker can hash candidates offline as fast as hardware allows. The length bound is what makes that infeasible; it is a security control, not input tidiness.
- **Compare in fixed time.** A normal string comparison stops at the first differing byte, and the timing difference leaks the expected value one byte at a time.

> **WHERE WE LANDED**
>
> PKCE is now recommended for *all* clients, including ones that do have a secret. A per-login proof defends against a stolen code even if the client secret itself has leaked, so the two mechanisms are complementary rather than alternatives.

8. Stage 7 — Asking a different question: OpenID Connect

STAGE 7

The flow answered "may this app act for the user". It never answered "who is the user".

Introduces: **id_token**, **nonce**, `scope=openid`

Everything so far solved **authorization**: Print Shop ended up holding an **access_token** that lets it call the photos API. Notice what it never obtained — any trustworthy statement about *who Alice is*.

That gap is easy to miss because the flow obviously involved a login. But look at what the access token actually is: a key addressed to the *resource server*. It says "the bearer may read photos". It does not say "the person who authorized this was Alice", it is not addressed to Print Shop, and Print Shop is not supposed to open it at all — to a client, an access token is an opaque string.

THE MISTAKE THIS CAUSES

The tempting shortcut is: "I got an access token, so I'll call `/userinfo` with it, and whoever it returns is my logged-in user." That is **not** a valid login, and it was a real and widespread vulnerability in the years before OIDC settled it. The next diagram shows why.

Notable unhappy flow: token substitution

An access token carries no binding to the application that received it. So a token obtained legitimately by *one* app can be presented to *another* app that treats it as proof of identity.

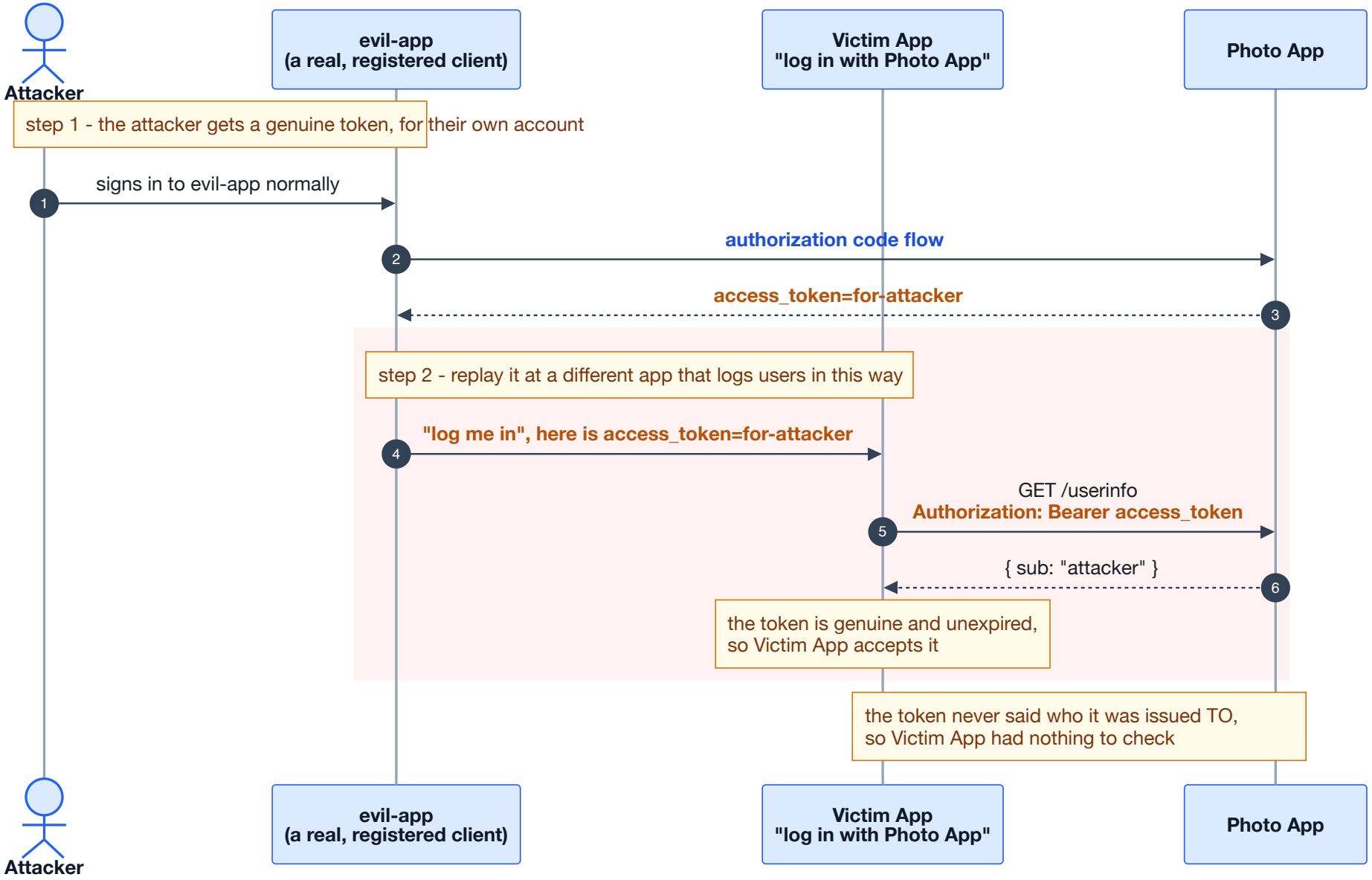

Figure 10 — Token substitution. Every message here is a valid token used in a valid way; the flaw is treating an authorization key as an identity assertion.

The reason this works is precise and worth stating exactly: **the access token has no audience the client can verify.** Fixing it needs a second artefact that is addressed to the client, and that is the whole idea of OpenID Connect.

What OIDC adds

OIDC is a thin identity layer on top of the same authorization code flow. The redirects are unchanged. The client adds `scope=openid` to the authorization request, and the provider returns one extra thing alongside the access token:

Artefact	What it is
access_token	A key **for the resource server**. Opaque to the client. Answers: *may the bearer call this API?*
id_token	A signed statement **for the client**, always a JWT. Answers: *who signed in, at which provider, and for which application?*

Both are amber — but they are not interchangeable, and the difference is the entire point:

- An access token is *used*; it is sent onward to an API. Never parse it, never trust its contents as a login.
- An id_token is *read*; it is never sent anywhere. It is your login receipt, and it must be validated on arrival.

The client validates an id_token by checking, at minimum:

- `iss` — issued by the provider I actually redirected to;
- `aud` — issued **for me**, matching my **client_id**. This is the check that defeats token substitution;
- `exp` — not expired;
- the **signature**, against the provider's published public key;
- `nonce` — matches the one I sent (below).

> **WHY DISCOVERY AND JWKS EXIST**
> Validating that signature requires the provider's public key, and knowing *which* key when providers rotate them. That is the entire reason for `/.well-known/openid-configuration` and `jwks_uri` — the same two endpoints your `WellKnownController` serves. They are OIDC machinery, which is why plain OAuth 2.0 has no equivalent.

The nonce, and the attack it stops

state tied the *redirect* to a request this client started. **nonce** does the analogous job one level down: it ties the *id_token itself* to this specific authorization request.

Without it, an attacker who captured a valid id_token — for their own account, from anywhere — could inject it into a victim's in-flight login and have the client accept it as the login result. The client generates a random nonce, sends it with the request, and the provider copies it into the id_token it mints. An id_token whose nonce does not match the one this browser sent is discarded.

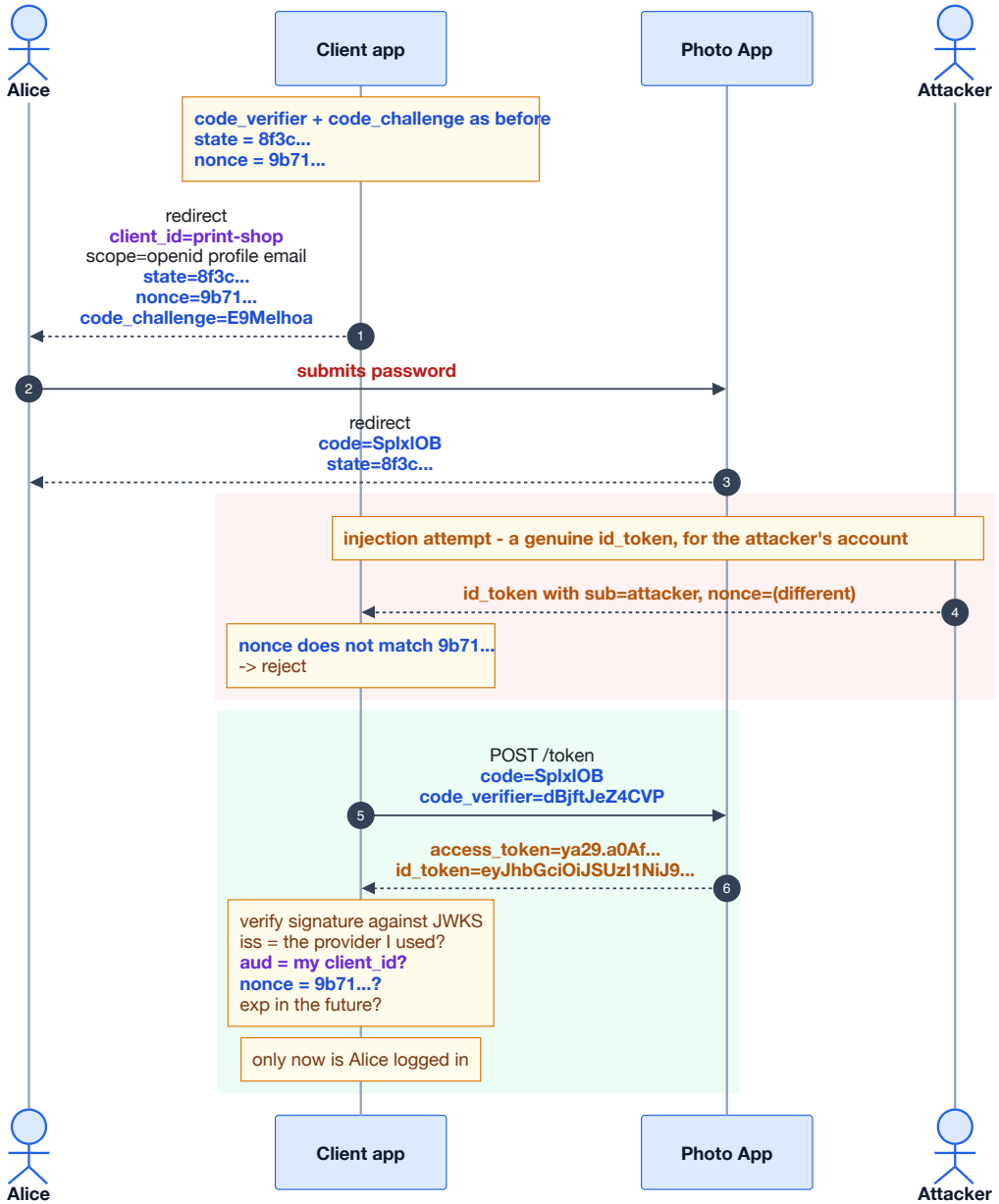

Figure 11 — OIDC on top of code + PKCE. The redirects are unchanged; the new work is the validation the client performs on arrival.

One more classic: trusting the token's own header

A JWT states its own signing algorithm in its header. Early libraries obeyed it, so an attacker could send `{"alg":"none"}` with no signature, or swap `RS256` for `HS256` and sign with the provider's *public* key — which is, after all, public. The rule is to decide the acceptable algorithm from configuration and reject anything else, never to let the token nominate how it should be verified.

> THE DISTINCTION TO CARRY AWAY
>
> **OAuth 2.0 answers "what may this app do". OpenID Connect answers "who is this user".** They are so often deployed together that people treat them as one protocol, but almost every identity bug in the wild comes from using an authorization answer to a question it was never asked.

9. The whole ladder in one table

Stage	Added	Because, without it…	Cost of getting it wrong
1	session_id	…the password would be sent on every single request	Guessable ids mean account takeover with no password
2	*the realisation that sharing the password cannot be made safe*		Every integration becomes a copy of your password
3	access_token	…third parties would need the password to act for the user	Bearer: anyone holding it is the user
3	redirect_uri (registered)	…an attacker names their own server and the provider posts the token to it	Loose matching has repeatedly become token theft
4	client_id	…the provider cannot tell which registration to enforce	Public by design; grants nothing alone
4	state	…a client accepts a callback it never initiated (login CSRF)	The victim's session silently binds to the attacker's account
5	authorization code	…the token itself has to travel through the address bar	Must be single-use and short-lived, or replay works
5	client_secret	…anyone holding a stolen code could redeem it	Impossible to keep in a browser or mobile app
6	code_verifier / code_challenge	…public clients have no way to prove the code is theirs	Skipping it on a public client means a stolen code is a full takeover
7	id_token	…the client has no verifiable statement of *who* logged in, only a key to an API	Using an access token as proof of login enables token substitution
7	nonce	…a captured id_token can be injected into someone else's in-flight login	Client accepts an identity assertion minted for a different session

10. Mapping it onto Cerberus and Synapse

Everything above is generic. Here is where each concept actually lives in your system.

Concept	In your codebase
password	Typed only into the Angular app on `:4200`, posted to `/User/login`, verified with BCrypt. Neither Synapse app ever receives it.
client_id	`synapse-spa`, registered via `POST /OAuth/clients`, stored in the `Client` table.
client_secret	Deliberately **empty**. That is what marks the client public: `string.IsNullOrEmpty(client.ClientSecret)` selects the PKCE path.
redirect_uri	`http://localhost:5173/auth/callback`, compared with an exact ordinal match at both the authorize and token steps.
state	Generated in `pkce.ts`, held in `sessionStorage`, checked in `exchangeCodeForTokens` before the code is redeemed.
code_verifier / **code_challenge**	Created in `pkce.ts` with `crypto.getRandomValues`; verified server-side in `Pkce.Verify`.
authorization code	Cached in Redis as `auth_code_{code}` with the challenge and redirect uri beside it, and deleted on redemption.
access_token	RS256 JWT, 15 minutes, validated by the Synapse API against the JWKS at `/.well-known/jwks.json`.
session	No server session for the SPA. The token *is* the session, which is why its short lifetime matters.
id_token / **nonce**	**Not implemented.** Cerberus serves an OIDC-shaped discovery document and a JWKS, but issues only an OAuth access token — there is no id_token and no nonce support. Stage 7 is therefore the part of this document your system has not reached yet.

One thing your system adds that the generic story does not have

Cerberus does not hand the OAuth parameters to its login page. It caches the whole authorization request server-side and redirects the browser with only an opaque `requestId`. The login page echoes that back and the server reconstitutes the request.

Read through the lens of this document, that is another application of Idea 1: the login page is given the least powerful thing that still does the job. It cannot alter the **redirect_uri** or the **code_challenge**, because it never holds them — so even a fully compromised login page cannot redirect the resulting code somewhere else.

> **WHERE TO GO NEXT**
>
> Two threads continue from here. **Refresh tokens**, and why they must rotate — a long-lived amber value is exactly the thing the whole ladder has been trying to avoid leaving lying around. And the **backend-for-frontend** pattern, which moves the code exchange server-side and puts the token in an `HttpOnly` cookie; that is the answer to the one weakness this design still has, namely a bearer token sitting in `localStorage` where any script on the origin can read it.